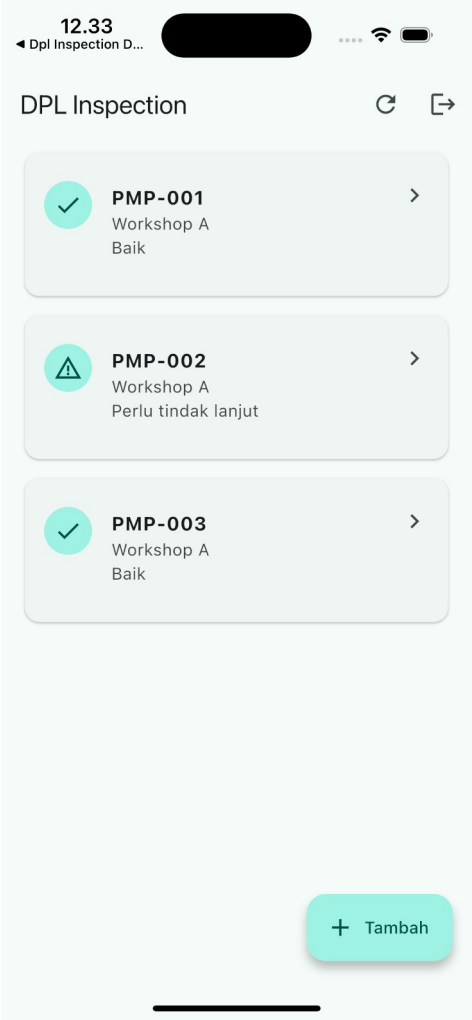
Materi Hari 2 Flutter Data dan State

Architecture, REST API, session dan cache | iPhone 16 Simulator

Trainer dan Pembuat Buku: Damar Galih Aji Pradana, S.Tr.Kom.

> **Proyek resmi:** `flutter_projects/hari_2_dpl_inspection_api`, hasil lengkap `flutter create` dengan Flutter 3.44.8. Proyek sudah memuat login, API, secure token, repository, ChangeNotifier, cache, form POST, retry, logout, dan test.



Hasil Hari 2 — data mock API pada iPhone 16

Target hari ini

Hubungkan UI checkpoint day-1 ke API tanpa mencampurkan request ke widget. Peserta membangun model JSON, service, repository, controller ChangeNotifier, login demo, secure storage dan cache. Fokus utamanya kemampuan mengikuti aliran data dan menangani kegagalan, bukan menggunakan banyak library.

Jadwal dan hasil

Waktu	Praktik	Bukti
08.30-10.00	Refactor layer dan jalankan mock server	Model/service/repository terpisah
10.15-12.00	Login, token, GET/POST dan parsing	Data server muncul; create berhasil
13.00-14.30	ChangeNotifier, session dan cache	State UI serta restart/logout
14.45-16.00	Search/filter, error, timeout, 401	AC kegagalan dan cache diuji
16.00-16.30	Review dan checkpoint	Tag day-2; bukti request

Aliran tanggung jawab

UI -> Controller -> Repository -> Service -> API. UI menerima aksi dan menampilkan state. Controller mengatur proses dan hasil. Repository menyediakan data serta kebijakan cache. Service membentuk request, membaca status, dan mengurai response. Model menjaga data aplikasi mempunyai tipe yang jelas.

Untuk program 3 hari, satu struktur sederhana cukup. Tidak perlu membuat interface/use-case pada setiap fungsi yang belum mempunyai kebutuhan nyata. Service/repository diberikan melalui constructor agar dapat diganti saat testing.

```
lib/
  main.dart
  models/inspection.dart
  services/api_service.dart
  repositories/inspection_repository.dart
  controllers/inspection_controller.dart
  storage/session_store.dart
  storage/inspection_cache.dart
  screens/login_screen.dart
  screens/inspection_list_screen.dart
  screens/inspection_form.dart
  widgets/inspection_tile.dart
```

Latihan 1: pindahkan model dan validator dari hari 1. Pisahkan data loading dari layar; pertahankan UI yang sudah bekerja. Jangan menulis ulang seluruh app hanya untuk mengganti struktur folder.

Menyiapkan API latihan

Folder praktik/mock_api berisi server Python tanpa dependency tambahan. Gunakan terminal kedua; server hidup selama kelas. Semua data dan token dummy. Jangan expose server ke internet.

```
python mock_api.py --host 127.0.0.1 --port 8000
```

Gunakan host 0.0.0.0 hanya bila physical device perlu mengakses LAN; batasi ke jaringan kelas dan periksa firewall. Endpoint /health harus memberi status ok sebelum Flutter diubah.

Target app	API_BASE_URL	Catatan
iPhone Simulator	http://localhost:8000	Pilihan praktik utama paket ini
Android Emulator resmi	http://10.0.2.2:8000	Mengarah ke host komputer emulator
Device Android	http://IP-LAN-KOMPUTER:8000	Device dan server punya route yang sesuai
Debug API lewat HTTPS	URL staging PIC	Pilihan yang sama dapat dipakai untuk release

```
flutter pub add http flutter_secure_storage shared_preferences
flutter run -d "iPhone 16"
--dart-define=API_BASE_URL=http://localhost:8000
```

Versi paket stabil yang kompatibel dipilih oleh pub resolver, kemudian pubspec.lock dikomit. Trainer memeriksa instruksi platform paket yang terpasang sebelum kelas; jangan memaksa downgrade/upgrade SDK tanpa membaca dependency conflict.

Izin jaringan Android

Tambahkan INTERNET sebagai child langsung manifest di android/app/src/main/AndroidManifest.xml. Bila debug memakai HTTP lokal, izinkan cleartext hanya pada debug manifest sesuai contoh paket. Jangan mengubah keamanan release agar mock HTTP dapat dipakai dalam production.

```
<uses-permission android:name="android.permission.INTERNET" />
```

Latihan 2: login /auth/login dengan trainer/training123 melalui curl/Postman dahulu. Catat response token dan status HTTP. GET /inspections membutuhkan Authorization Bearer. File request siap pakai ada pada requests.http.

Model JSON dan request GET

JSON object/list berbeda dari object Dart. Parsing adalah batas penting: field yang hilang atau berubah harus menghasilkan error yang dapat ditangani, bukan null yang baru crash jauh di UI.

```
factory Inspection.fromJson(Map<String, dynamic> json) {
  return Inspection(
    id: json['id'] as String,
    assetCode: json['assetCode'] as String,
    location: json['location'] as String,
    condition: json['condition'] as String,
    note: (json['note'] as String?) ?? '',
    createdAt: DateTime.parse(json['createdAt'] as String).toUtc(),
  );
}
```

Untuk kontrak dummy yang stabil, cast eksplisit cukup untuk menunjukkan tipe yang diharapkan. Saat integrasi backend nyata, petakan error parsing serta validasi field sesuai kontrak backend. Jangan mengabaikan field wajib lalu menampilkan data keliru.

```
final response = await client.get(
  Uri.parse('$baseUrl/inspections?per_page=50'),
  headers: {'Authorization': 'Bearer $token'},
).timeout(const Duration(seconds: 10));
if (response.statusCode != 200) {
  throw ApiException(response.statusCode, 'Daftar gagal dimuat');
}
final body = jsonDecode(response.body) as Map<String, dynamic>;
final items = (body['items'] as List)
    .map((e) => Inspection.fromJson(e as Map<String, dynamic>))
    .toList();
```

Potongan berada dalam service yang menerima http.Client melalui constructor. Close client saat pemilik service dilepas. API_BASE_URL dibaca satu kali melalui String.fromEnvironment, divalidasi tidak kosong, dan bukan disalin sebagai literal di banyak layar.

Praktik 3

Bangun service login/list/create; repository mengembalikan model. Controller load dijalankan saat layar pertama dibuat atau saat retry. Tambahkan breakpoint pada response, parsing, dan assignment controller. Periksa apakah UI menunjukkan error yang berbeda dari empty.

Expected result: record server tampil dan model dapat diperiksa tanpa UI. Data hari 1 tetap menjadi seed/reference, bukan sumber utama setelah integrasi selesai.

POST, token dan pemetaan error

POST mengirim field form sebagai JSON. Validasi client menjaga pengalaman pengguna; validasi server tetap menjadi sumber penerimaan data. UI hanya menambah item setelah server mengembalikan object dengan ID server.

```
final response = await client.post(
  Uri.parse('$baseUrl/inspections'),
  headers: {'Content-Type': 'application/json',
    'Authorization': 'Bearer $token'},
  body: jsonEncode({'assetCode': assetCode.trim(),
    'location': location.trim(), 'condition': condition,
    'note': note.trim()}),
).timeout(const Duration(seconds: 10));
```

Kejadian	Perilaku app	Hal yang tidak boleh dilakukan
201 create	Parse object; close form; refresh daftar	Membuat ID baru lagi di client
401	Hapus token/cache; buka	Membaca cache lalu berpura-

Kejadian	Perilaku app	Hal yang tidak boleh dilakukan
	login	pura masih login
422	Tampilkan pesan; pertahankan form	Menghapus input pengguna
500/network	Error dan retry yang dipilih pengguna	Retry POST otomatis tanpa idempotency
Timeout	Beri pesan bahwa hasil belum pasti	Menganggap server pasti tidak menerima data
JSON invalid	Error parsing tercatat tanpa token	Mengganti semuanya menjadi daftar kosong

Latihan 4

1. Cegah isSubmitting true menerima request kedua.
2. Periksa semua validator sebelum controller.create dipanggil.
3. Selama POST berjalan, disable tombol simpan dan tampilkan label proses.
4. Pada 422/500, form tetap terbuka dengan nilai awal. Pada sukses, kembali dan load daftar server.
5. Simulasikan 401 memakai token salah; app kembali ke login dan cache hilang.

Interceptors: konsep titik bersama untuk menambahkan auth header serta memetakan error. Dalam kelas ini http.Client dan satu ApiService cukup. Dio/interceptor dapat diperkenalkan sebagai alternatif bila tim sudah memakainya, bukan dipasang sebagai state/network stack kedua.

Keamanan session: password demo hanya dipakai untuk request login. Token tidak disimpan pada source, log atau SharedPreferences. Mock token tidak mempunyai expiry/refresh production; peserta tidak boleh menganggapnya rancangan auth operasional.

Satu state management sederhana

ChangeNotifier adalah mekanisme notifikasi. Controller mengubah data, lalu notifyListeners memberi sinyal UI untuk membangun ulang. Ini tidak otomatis menangani request, cache, race condition atau error; semua kebijakan tersebut tetap harus dirancang.

```
class InspectionController extends ChangeNotifier {
  final InspectionRepository repository;
  List<Inspection> items = [];
  bool isLoading = false;
  String? errorMessage;
  InspectionController(this.repository);

  Future<void> load() async {
    if (isLoading) return;
    isLoading = true;
    errorMessage = null;
    notifyListeners();
    try {
```

```
      items = await repository.list();
    } catch (e) {
      errorMessage = 'Gagal mengambil data. Coba lagi.';
    } finally {
      isLoading = false;
      notifyListeners();
    }
  }
}
```

Ini contoh konsep; tambahkan perlindungan disposed, 401, cache metadata, dan submit guard seperti skeleton controller di folder praktik. Pemilik controller melepasnya di dispose dan tidak membuatnya kembali setiap build.

Menghubungkan UI

```
ListenableBuilder(
  listenable: controller,
  builder: (context, _) {
    if (controller.isLoading) {
      return const Center(child: CircularProgressIndicator());
    }
    return buildResult(controller);
  },
)
```

Latihan 5: render success, empty, error/retry dan cache melalui state controller. State form seperti selection boleh tetap memakai setState. Jangan memindahkan setiap karakter input ke global state hanya karena ChangeNotifier sudah digunakan.

Review: controller tidak menerima BuildContext; layar memutuskan navigation. Callback session expired dikirim ke pemilik session. Perubahan state mudah ditelusuri dari aksi UI sampai service.

Session dan cache yang jujur

Secure storage menyimpan token melalui mekanisme platform. SharedPreferencesAsync dipakai untuk cache nonrahasia latihan. Cache bukan database transaksi dan bukan sumber kebenaran saat server dapat dijangkau.

```
const secure = FlutterSecureStorage();
await secure.write(key: 'session_token', value: token);
final storedToken = await secure.read(key: 'session_token');
await secure.delete(key: 'session_token');
```

```
final prefs = SharedPreferencesAsync();
await prefs.setString('inspection_cache', jsonEncode({
  'fetchedAt': DateTime.now().toUtc().toIso8601String(),
  'items': items.map((e) => e.toJson()).toList(),
}));
```

Kebijakan yang dipraktikkan

1. Startup membaca token; bila ada, GET tetap memvalidasi session server.
2. GET sukses memperbarui items dan cache. fetchedAt menunjukkan saat data didapat, bukan saat cache dibaca.
3. Hanya network failure/timeout yang boleh memicu fallback cache. Response 401/422/500 atau JSON invalid tidak disamarkan sebagai offline.
4. Cache invalid diabaikan/dihapus. Tanpa cache, tampilkan error/retry.
5. Cache tampil read-only dengan label ‘Data cache’ serta fetchedAt; tombol tambah dinonaktifkan.
6. Logout/401 membersihkan token dan cache sebelum kembali ke login.

Batas: fallback session offline setelah restart hanya demonstrasi latihan; keberadaan token lokal tidak membuktikan authorization server saat ini. App production perlu kebijakan expiry, akses offline, proteksi data dan retensi dari pemilik sistem.

Latihan 6

GET sukses, hentikan server, lalu refresh. Lihat cache dan label waktunya. Hapus cache dan ulangi; app harus menunjukkan error. Rusakkan JSON cache; app tidak crash. Nyalakan server, ubah token menjadi salah; app logout, bukan fallback cache.

Pengayaan: gunakan local database saat data terstruktur dan query kompleks dibutuhkan. Antrean write offline memerlukan idempotency/sync/conflict handling dan tidak cukup dengan menyimpan POST ke preferences.

Search, filtering dan debugging API

Hari ini search/filter dilakukan pada items yang sudah dimuat agar scope jelas. Gunakan query lower-case terhadap assetCode/location dan predicate kondisi. Jika server mempunyai pagination, search lokal hanya mencari halaman yang sudah dimuat; label/perilaku harus sesuai, atau pindahkan search ke API pada pengayaan.

```
final query = search.trim().toLowerCase();
final visible = items.where((item) {
  final matchesText = item.assetCode.toLowerCase().contains(query)
      || item.location.toLowerCase().contains(query);
  return matchesText && (filter == null || item.condition == filter);
}).toList();
```

Pagination demo: mock mendukung page/per_page. Trainer membandingkan dua halaman dan total. Implementasi load-more harus menjaga urutan, mencegah request ganda dan deduplicate berdasarkan id; ini bukan acceptance inti hari 2.

Gejala	Pemeriksaan	Perbaikan
Connection refused	Server/port/host dan jenis simulator	iPhone: localhost; Android: 10.0.2.2
Device timeout	Route LAN dan firewall	Uji dari device; host server sesuai

Gejala	Pemeriksaan	Perbaikan
Cleartext blocked	HTTP debug config	Atur debug-only atau HTTPS
GET 401	Header/token/endpoint	Periksa auth tanpa mencetak token
Type cast error	Shape items dan nama field	Cocokkan kontrak, bukan force null
UI tidak berubah	notifyListeners/pemilik controller	Periksa flow controller ke builder
API dipanggil terus	Request dari build	Pindahkan ke init/controller lifecycle

Evaluasi dan checkpoint

Jelaskan perbedaan service/repository/controller, mengapa POST timeout belum tentu gagal di server, serta mengapa 401 tidak memakai cache. Jalankan AC1-AC11 PRD Hari 2, lalu commit/tag day-2. Simpan bukti tanpa authorization header/password.

Referensi resmi: https://docs.flutter.dev/app-architecture/recommendations ; https://docs.flutter.dev/cookbook/networking ; https://docs.flutter.dev/data-and-backend/state-mgmt/simple ; https://pub.dev/packages/flutter_secure_storage ; https://pub.dev/packages/shared_preferences . Diperiksa 03 Oktober 2026. Skeleton kode perlu dirangkai peserta dan diverifikasi dengan analyze/test pada SDK kelas; bukan aplikasi production siap rilis.